메트릭 수집 시스템 분석

개념적 배경

왜 메트릭 수집이 필요한가?

메트릭 수집 시스템은 애플리케이션의 성능, 상태, 동작을 정량적으로 측정하고 모니터링하는 시스템입니다. geth에서는 다음과 같은 목적으로 사용됩니다:

1. **실시간 모니터링**: 시스템 상태의 지속적 감시
2. **성능 추적**: 처리량, 응답 시간, 리소스 사용량 측정
3. **이상 탐지**: 비정상적인 패턴이나 임계값 초과 감지
4. **용량 계획**: 리소스 사용 트렌드 분석을 통한 확장 계획
5. **SLA 관리**: 서비스 수준 협약 준수 여부 확인
6. **문제 진단**: 성능 저하나 장애 원인 분석

다른 접근법과의 비교

| 접근법 | 장점 | 단점 | 사용 사례 |

|--------|------|------|----------|

| **Pull 기반 (Prometheus)** | 중앙 집중식, 안정적 | 네트워크 오버헤드 | 마이크로서비스 |

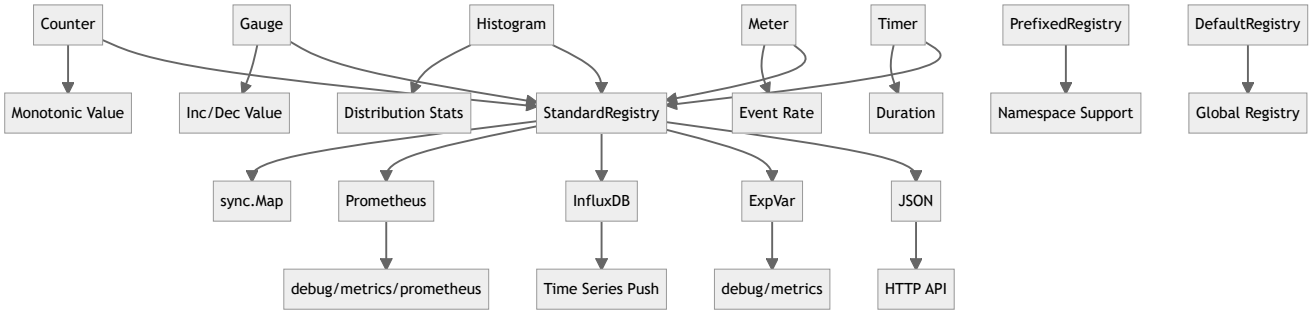
| **Push 기반 (InfluxDB)** | 실시간성, 유연성 | 복잡한 설정 | IoT, 실시간 분석 |

| **로그 기반** | 상세한 정보 | 파싱 복잡성 | 디버깅, 감사 |

| **APM 도구** | 종합적 분석 | 높은 비용 | 엔터프라이즈 |

구현 분석

핵심 아키텍처



주요 컴포넌트

1. 메트릭 타입별 구현

Counter (카운터)

```
type Counter atomic.Int64
```

```
func (c *Counter) Inc(i  int64) {
(*atomic.Int64)(c).Add(i)
}
```

```
func (c *Counter) Dec(i  int64) {
(*atomic.Int64)(c).Add(-i)
}
```

```go
func (c *Counter) Snapshot() CounterSnapshot {
return  CounterSnapshot((*atomic.Int64)(c).Load())
}
```

특징:

- 원자적 연산으로 동시성 안전
- 단조 증가하는 값 (요청 수, 에러 수 등)
- 메모리 효율적 (8바이트)

Gauge (게이지)

```go
type Gauge atomic.Int64

func (g *Gauge) Update(v  int64) {
(*atomic.Int64)(g).Store(v)
}

func (g *Gauge) UpdateIfGt(v  int64) {
value := (*atomic.Int64)(g)
for {
exist := value.Load()
if  exist >= v {
break
}
```

```
if value.CompareAndSwap(exist, v) {
break
}
}
}
```

특징:

- 임의로 증감하는 값 (메모리 사용량, 연결 수 등)
- Compare-and-Swap을 통한 조건부 업데이트
- 실시간 상태 반영

Histogram (히스토그램)

```
type StandardHistogram struct {
sample Sample
}

func (h *StandardHistogram) Update(v int64) {
h.sample.Update(v)
}

func (h *StandardHistogram) Snapshot() HistogramSnapshot {
return h.sample.Snapshot()
}
```

특징:

- 값의 분포 통계 계산
- 백분위수 (percentile) 제공
- 샘플링을 통한 메모리 사용량 제한

Meter (미터)

```
type Meter struct {
count atomic.Int64
uncounted atomic.Int64
rateMean atomic.Uint64
a1, a5, a15 *EWMA // 지수 가중 이동 평균
startTime time.Time
stopped atomic.Bool
}

func (m *Meter) Mark(n  int64) {
m.uncounted.Add(n)
}

func (m *Meter) tick() {
n := m.uncounted.Swap(0)
count := m.count.Add(n)
// 평균 비율 계산
```

```
m.rateMean.Store(math.Float64bits(
float64(count) / time.Since(m.startTime).Seconds()))
// EWMA 업데이트
m.a1.Update(n)
m.a5.Update(n)
m.a15.Update(n)
}
```

특징:

- 이벤트 발생률 측정 (TPS, 처리량 등)
- 1분, 5분, 15분 이동 평균 제공
- 백그라운드 고루틴에서 주기적 업데이트

Timer (타이머)

```
type Timer struct {
histogram Histogram
meter *Meter
}

func (t *Timer) Update(d time.Duration) {
t.histogram.Update(int64(d))
t.meter.Mark(1)
}
```

```
func (t *Timer) Time(f func()) {
ts := time.Now()
f()
t.Update(time.Since(ts))
}
```

특징:

- 시간 측정과 통계 제공
- 히스토그램과 미터의 조합
- 함수 실행 시간 자동 측정

2. 레지스트리 시스템

StandardRegistry

```
type StandardRegistry struct {
metrics sync.Map
}

func (r *StandardRegistry) GetOrRegister(name string, ctor func() interface{})
// 빠른 경로: 이미 존재하는 경우
cached, ok := r.metrics.Load(name)
if ok {
return cached
}
```

```
// 느린 경로: 새로 생성
item, _, _ := r.loadOrRegister(name, ctor())
return item
}

func (r *StandardRegistry) Each(f func(string, interface{})) {
for name, i := range r.registered() {
f(name, i)
}
}
```

특징:

- `sync.Map` 기반으로 높은 동시성 성능
- 지연 초기화 지원
- 스레드 안전한 등록/해제

PrefixedRegistry

```
type PrefixedRegistry struct {
underlying Registry
prefix string
}

func (r *PrefixedRegistry) Register(name string, metric interface{}) error {
```

```
realName := r.prefix + name

return  r.underlying.Register(realName, metric)

}
```

특징:

- 네임스페이스 지원
- 계층적 메트릭 구조
- 모듈별 메트릭 분리

3. 시스템 메트릭 수집

프로세스 메트릭 수집기

```
func  CollectProcessMetrics(refresh time.Duration) {

var (

cpuSysLoad = GetOrRegisterGauge("system/cpu/sysload", DefaultRegistry)

cpuProcLoad = GetOrRegisterGauge("system/cpu/procload", DefaultRegistry)

memTotal = GetOrRegisterGauge("system/memory/held", DefaultRegistry)

heapUsed = GetOrRegisterGauge("system/memory/used", DefaultRegistry)

diskReads = GetOrRegisterMeter("system/disk/readcount", DefaultRegistry)

diskWrites = GetOrRegisterMeter("system/disk/writecount", DefaultRegistry)

)

for {

// CPU 통계 수집

ReadCPUStats(&cpustats[now])

sysLoad := cpustats[now].GlobalTime - cpustats[prev].GlobalTime
```

```
procLoad := cpustats[now].LocalTime - cpustats[prev].LocalTime

cpuSysLoad.Update(int64(sysLoad / secondsSinceLastCollect * 100))

cpuProcLoad.Update(int64(procLoad / secondsSinceLastCollect * 100))

// 메모리 통계 수집

readRuntimeStats(&rstats[now])

memTotal.Update(int64(rstats[now].MemTotal))

heapUsed.Update(int64(rstats[now].MemTotal - rstats[now].HeapUnused))

// 디스크 통계 수집

ReadDiskStats(&diskstats[now])

diskReads.Mark(diskstats[now].ReadCount - diskstats[prev].ReadCount)

diskWrites.Mark(diskstats[now].WriteCount - diskstats[prev].WriteCount)

time.Sleep(refresh)

}

}
```

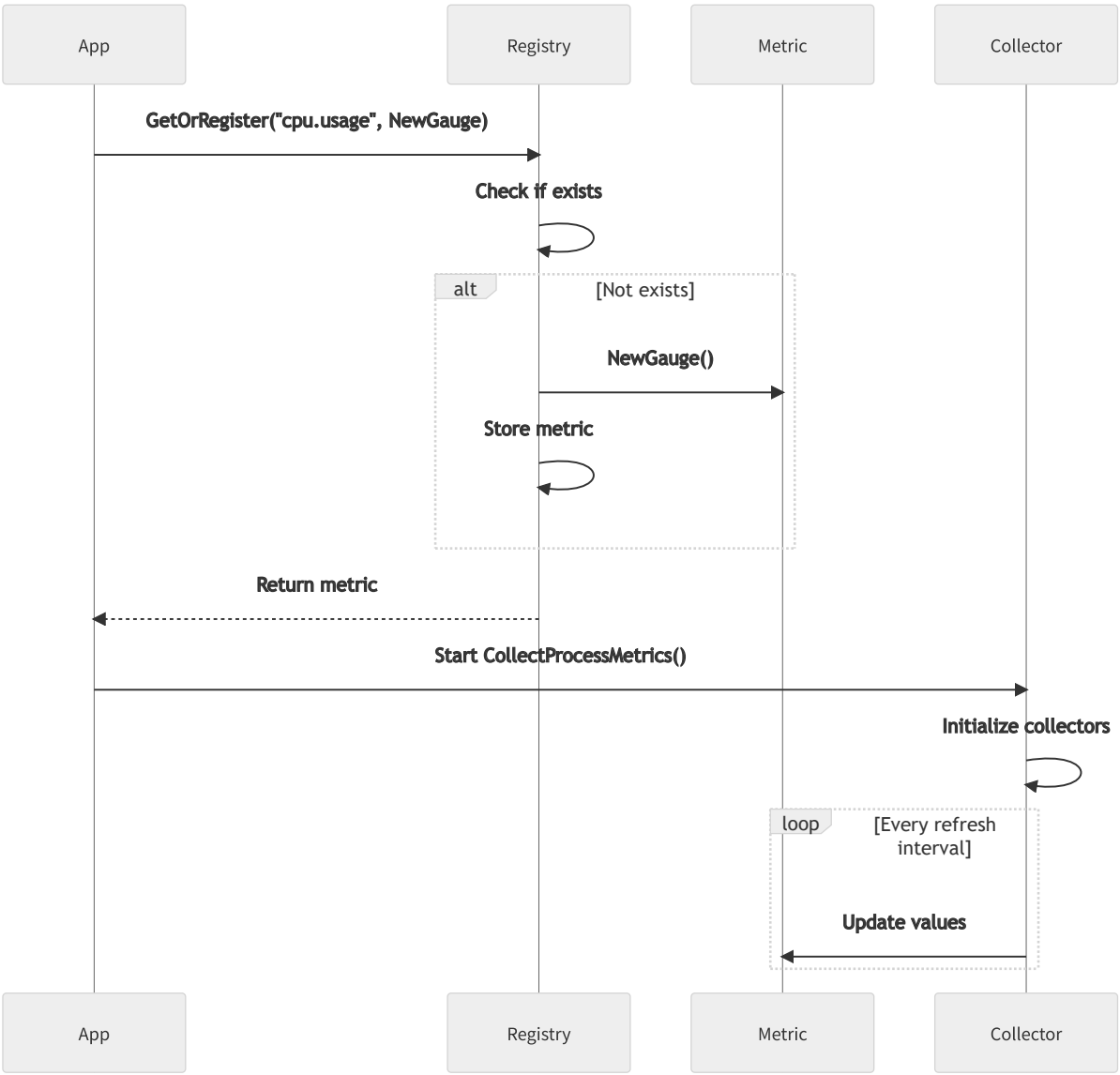
런타임 메트릭 수집

```
var  runtimeSamples = []metrics.Sample{

{Name: "/gc/pauses:seconds"}, // GC 일시정지 시간

{Name: "/gc/heap/allocs:bytes"}, // 힙 할당량

{Name: "/gc/heap/frees:bytes"}, // 힙 해제량

{Name: "/memory/classes/total:bytes"}, // 총 메모리 사용량

{Name: "/sched/goroutines:goroutines"}, // 고루틴 수
```

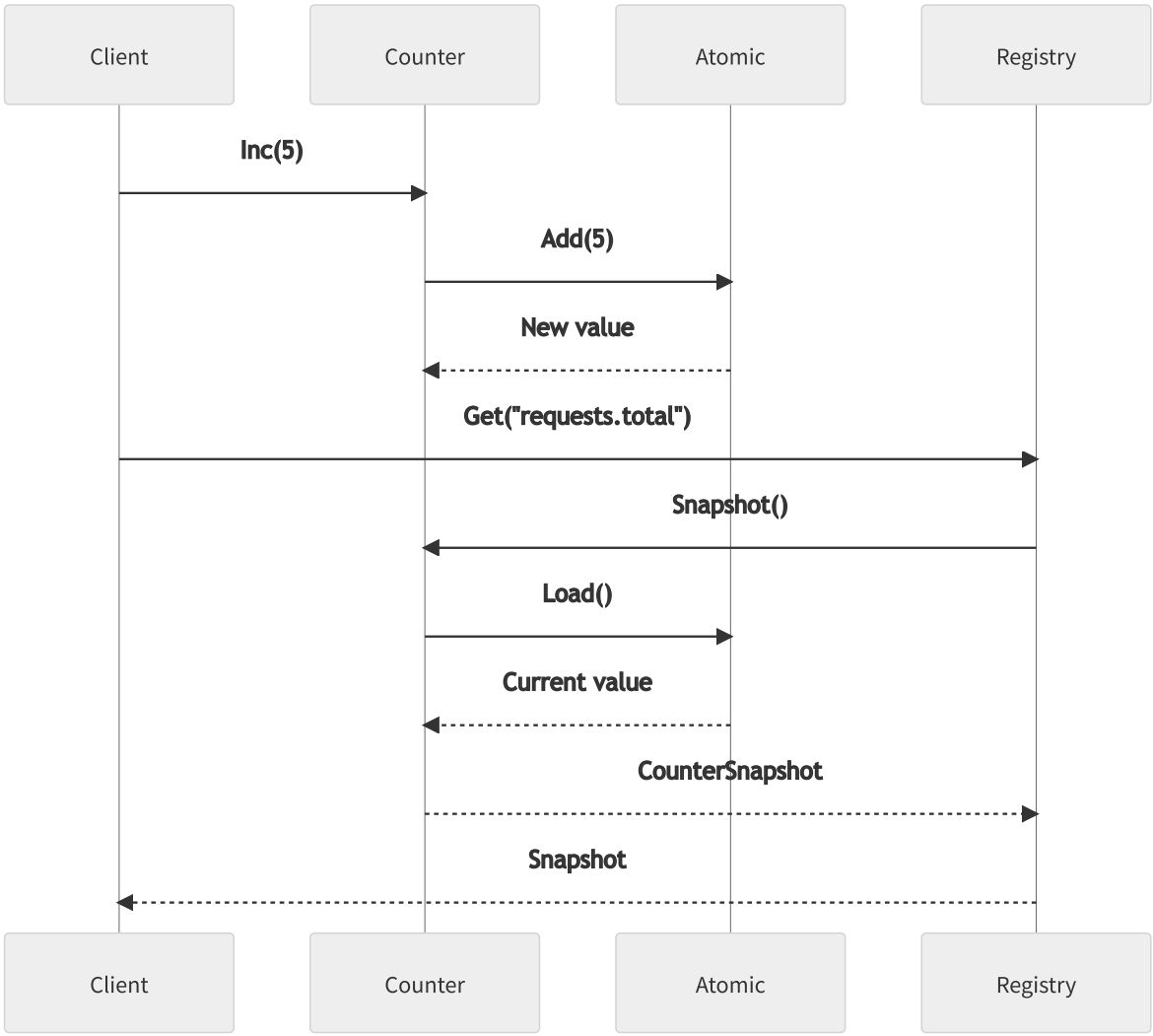
```
{Name: "/sched/latencies:seconds"}, // 스케줄링 지연시간
}


func readRuntimeStats(v *runtimeStats) {
metrics.Read(runtimeSamples)
for _, s := range runtimeSamples {
if s.Value.Kind() == metrics.KindBad {
continue
}
switch s.Name {
case "/gc/pauses:seconds":
v.GCPauses = s.Value.Float64Histogram()
case "/memory/classes/total:bytes":
v.MemTotal = s.Value.Uint64()
case "/sched/goroutines:goroutines":
v.Goroutines = s.Value.Uint64()
}
}
}
```

실행 흐름

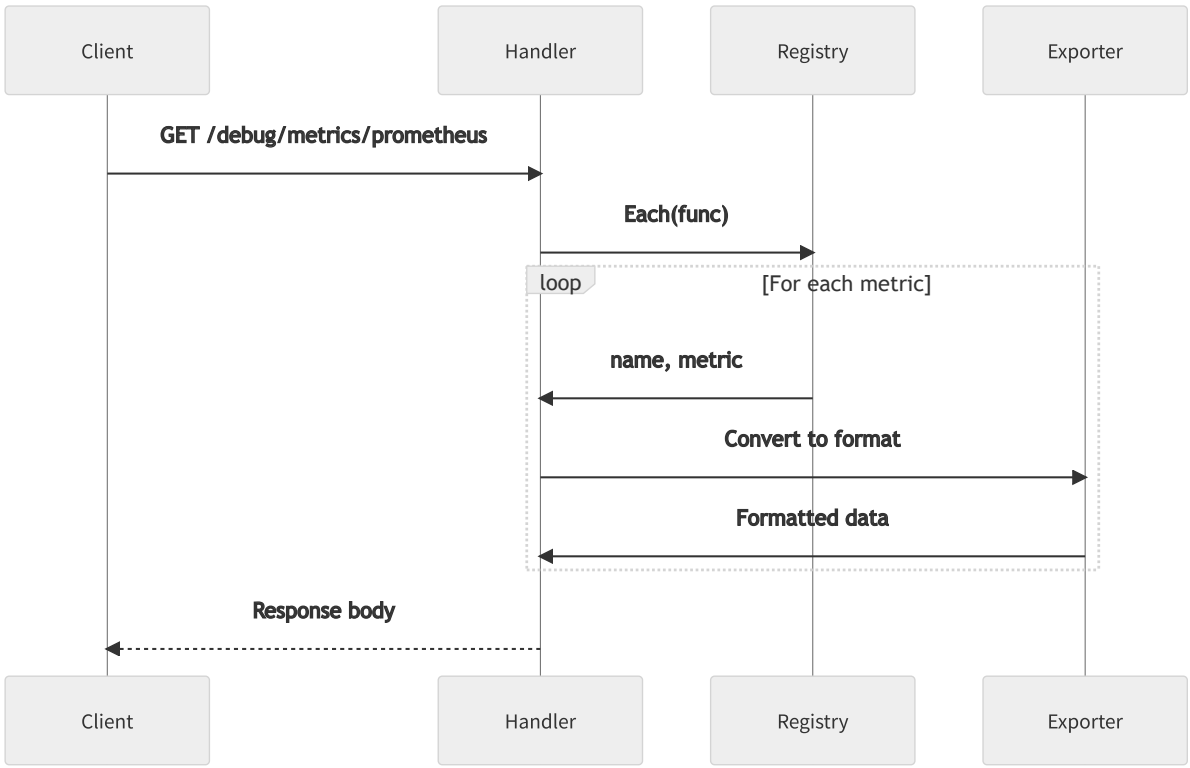
1. 메트릭 등록 및 초기화


App
Registry
Metric
Collector
GetOrRegister("cpu.usage", NewGauge)
Check if exists
alt
[Not exists]
NewGauge()
Store metric
Return metric
Start CollectProcessMetrics()
Initialize collectors
loop
[Every refresh interval]
Update values
App
Registry
Metric
Collector

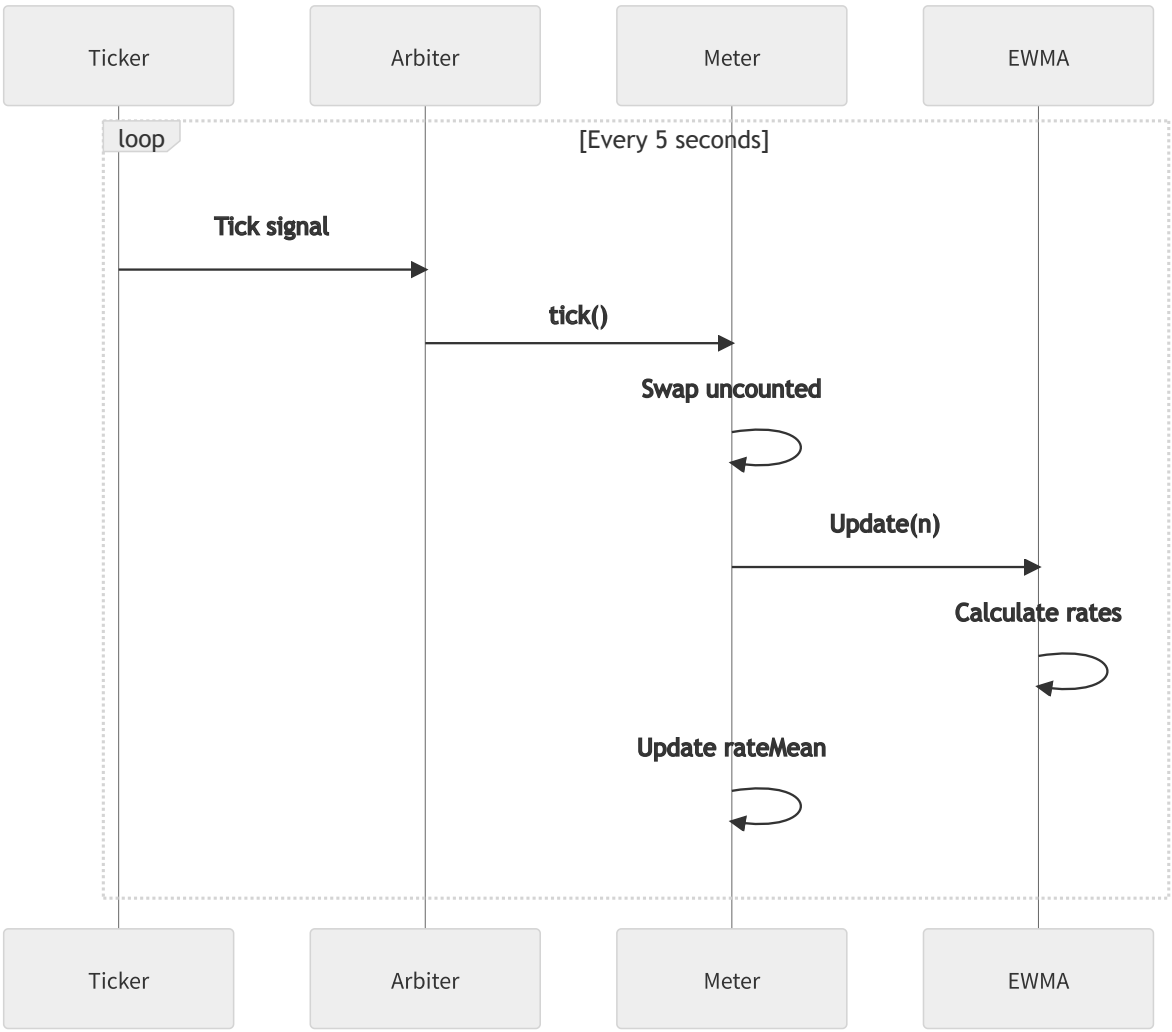
2. 메트릭 업데이트 흐름


Client
Counter
Atomic
Registry
Inc(5)
Add(5)
New value
Get("requests.total")
Snapshot()
Load()
Current value
CounterSnapshot
Snapshot
Client
Counter
Atomic
Registry

3. 메트릭 내보내기 흐름


Client
Handler
Registry
Exporter
GET /debug/metrics/prometheus
Each(func)
loop
[For each metric]
name, metric
Convert to format
Formatted data
Response body
Client
Handler
Registry
Exporter

4. 미터 틱 처리 흐름



내보내기 인터페이스 분석

1. Prometheus 형식

메트릭 변환

```
func (c *collector) Add(name  string, i  interface{}) error {
switch  metric := i.(type) {
```

```
case *Counter:
c.writeCounter(name, metric.Snapshot())
case *Gauge:
c.writeGauge(name, metric.Snapshot())
case Histogram:
c.writeHistogram(name, metric.Snapshot())
case *Meter:
c.writeMeter(name, metric.Snapshot())
case *Timer:
c.writeTimer(name, metric.Snapshot())
}
return  nil
}


func (c *collector) writeCounter(name  string, snapshot CounterSnapshot) {
c.buff.WriteString("# TYPE ")
c.buff.WriteString(name)
c.buff.WriteString(" counter\n")
c.buff.WriteString(name)
c.buff.WriteString(" ")
c.buff.WriteString(strconv.FormatInt(snapshot.Count(), 10))
c.buff.WriteString("\n")
```

```
}
```

출력 형식

```
# TYPE geth_requests_total counter
geth_requests_total 12345

# TYPE geth_memory_used gauge
geth_memory_used 1073741824

# TYPE geth_response_time histogram
geth_response_time_bucket{le="0.1"} 100
geth_response_time_bucket{le="0.5"} 200
geth_response_time_bucket{le="1.0"} 300
geth_response_time_bucket{le="+Inf"} 350
geth_response_time_sum 175.5
geth_response_time_count 350
```

2. InfluxDB 형식

메트릭 변환

```go
func readMeter(namespace, name string, i interface{}) (string, map[string]int
switch metric := i.(type) {
case *Counter:
```

```
measurement := fmt.Sprintf("%s%s.count", namespace, name)
fields := map[string]interface{}{
"value": metric.Snapshot().Count(),
}
return  measurement, fields
case Histogram:
ms := metric.Snapshot()
ps := ms.Percentiles([]float64{0.25, 0.5, 0.75, 0.95, 0.99})
measurement := fmt.Sprintf("%s%s.histogram", namespace, name)
fields := map[string]interface{}{
"count": ms.Count(),
"max": ms.Max(),
"mean": ms.Mean(),
"min": ms.Min(),
"p25": ps[0],
"p50": ps[1],
"p75": ps[2],
"p95": ps[3],
"p99": ps[4],
}
return  measurement, fields
}
return  "", nil
```

```
}
```

출력 형식

```
geth.requests.count value=12345 1609459200000000000

geth.memory.gauge value=1073741824 1609459200000000000

geth.response_time.histogram count=350,max=5000,mean=501.4,min=10,p50=450,p95=12
```

3. ExpVar 형식

JSON 출력

```
func (exp *exp) expHandler(w http.ResponseWriter, r *http.Request) {

exp.syncToExpvar()

w.Header().Set("Content-Type", "application/json; charset=utf-8")

fmt.Fprintf(w, "{\n")

first := true

expvar.Do(func(kv expvar.KeyValue) {

if !first {

fmt.Fprintf(w, ",\n")

}

first = false

fmt.Fprintf(w, "%q: %s", kv.Key, kv.Value)
```

```
})

fmt.Fprintf(w, "\n}\n")

}
```

출력 형식

```
{

"geth.requests.total": 12345,

"geth.memory.used": 1073741824,

"geth.response_time.count": 350,

"geth.response_time.mean": 501.4,

"geth.response_time.95-percentile": 1200

}
```

성능 최적화

1. 메모리 효율성

원자적 연산 사용

```
// 뮤텍스 대신 원자적 연산 사용

type Counter atomic.Int64


func (c *Counter) Inc(i int64) {

(*atomic.Int64)(c).Add(i) // 락 없는 연산
```

```
}
```

샘플링을 통한 메모리 제한

```
type UniformSample struct {
count int64
values []int64
reservoir []int64
}

func (s *UniformSample) Update(v int64) {
s.count++
if s.count <= reservoirSize {
s.values[s.count-1] = v
} else {
r := rand.Int63n(s.count)
if r < reservoirSize {
s.values[r] = v
}
}
}
```

2. 동시성 최적화

sync.Map 활용

```
type StandardRegistry struct {
metrics sync.Map // 읽기 최적화된 맵
}

func (r *StandardRegistry) Get(name  string) interface{} {
item, _ := r.metrics.Load(name) // 락 없는 읽기
return  item
}
```

배치 업데이트

```
func (m *Meter) tick() {
// 누적된 값을 한 번에 처리
n := m.uncounted.Swap(0)
if  n == 0 {
return
}
count := m.count.Add(n)
// 여러 EWMA를 배치로 업데이트
m.a1.Update(n)
m.a5.Update(n)
m.a15.Update(n)
```

```
}
```

3. 조건부 메트릭 수집

활성화 플래그 확인

```
var metricsEnabled = false

func expensiveMetricUpdate() {
if !metrics.Enabled() {
return // 비활성화 시 즉시 반환
}
// 비용이 높은 메트릭 업데이트
}
```

지연 초기화

```
func GetOrRegisterHistogram(name string, r Registry, s Sample) Histogram {
return getOrRegister(name, func() Histogram {
return NewHistogram(s)
}, r)
}
```

실시간 모니터링 및 알림 시스템

1. 임계값 기반 알림

메트릭 임계값 설정

```
type AlertRule struct {
MetricName string
Threshold float64
Operator string // >, <, ==, !=
Duration time.Duration
}

func checkAlerts(registry Registry, rules []AlertRule) {
registry.Each(func(name string, metric interface{}) {
for _, rule := range rules {
if rule.MetricName == name {
if evaluateRule(metric, rule) {
sendAlert(rule, metric)
}
}
}
})
}
```

알림 전송

```
func sendAlert(rule AlertRule, metric interface{}) {
alert := Alert{
MetricName: rule.MetricName,
Value: getCurrentValue(metric),
Threshold: rule.Threshold,
Timestamp: time.Now(),
}
// Slack, Email, PagerDuty 등으로 전송
notificationService.Send(alert)
}
```

2. 대시보드 구성

Grafana 대시보드 설정

```
{
"dashboard": {
"title": "Geth Metrics",
"panels": [
{
"title": "CPU Usage",
"type": "graph",
"targets": [
{
```

```
"expr": "geth_system_cpu_procload",
"legendFormat": "Process CPU"
}
]
},
{
"title": "Memory Usage",
"type": "graph",
"targets": [
{
"expr": "geth_system_memory_used",
"legendFormat": "Used Memory"
}
]
}
]
}
}
```

실제 사용 예시

1. 기본 메트릭 수집 설정

geth 시작 시 메트릭 활성화

```
# Prometheus 메트릭 활성화
geth --metrics --metrics.addr=0.0.0.0 --metrics.port=6060

# InfluxDB 연동
geth --metrics \
--metrics.influxdb \
--metrics.influxdb.endpoint=http://influxdb:8086 \
--metrics.influxdb.database=geth \
--metrics.influxdb.username=admin \
--metrics.influxdb.password=password
```

커스텀 메트릭 추가

```go
var (
requestCounter = metrics.GetOrRegisterCounter("api/requests/total", nil)
responseTimer = metrics.GetOrRegisterTimer("api/response/time", nil)
errorGauge = metrics.GetOrRegisterGauge("api/errors/current", nil)
)

func handleRequest(w http.ResponseWriter, r *http.Request) {
requestCounter.Inc(1)
timer := time.Now()
defer func() {
```

```
responseTimer.Update(time.Since(timer))
}()
// 요청 처리 로직
if err := processRequest(r); err != nil {
errorGauge.Inc(1)
http.Error(w, err.Error(), 500)
return
}
errorGauge.Dec(1)
w.WriteHeader(200)
}
```

2. 프로덕션 모니터링 설정

Docker Compose 설정

```
version: '3.8'
services:
geth:
image: ethereum/client-go:latest
command: >
--metrics
--metrics.addr=0.0.0.0
--metrics.port=6060
```

```
--metrics.influxdb

--metrics.influxdb.endpoint=http://influxdb:8086

ports:

- "8545:8545"

- "6060:6060"

prometheus:

image: prom/prometheus:latest

ports:

- "9090:9090"

volumes:

- ./prometheus.yml:/etc/prometheus/prometheus.yml

grafana:

image: grafana/grafana:latest

ports:

- "3000:3000"

environment:

- GF_SECURITY_ADMIN_PASSWORD=admin
```

Prometheus 설정

```
global:

scrape_interval: 15s
```

```
scrape_configs:

- job_name: 'geth'

static_configs:

- targets: ['geth:6060']

metrics_path: '/debug/metrics/prometheus'

scrape_interval: 5s
```

3. 자동화된 모니터링

헬스체크 스크립트

```
#!/bin/bash

# geth 상태 모니터링 스크립트


METRICS_URL="http://localhost:6060/debug/metrics"

ALERT_THRESHOLD_CPU=80

ALERT_THRESHOLD_MEMORY=8589934592  # 8GB


# 메트릭 수집

CPU_USAGE=$(curl  -s  $METRICS_URL | jq  -r  '.["system/cpu/procload"]')

MEMORY_USAGE=$(curl  -s  $METRICS_URL | jq  -r  '.["system/memory/used"]')

PEER_COUNT=$(curl  -s  $METRICS_URL | jq  -r  '.["p2p/peers"]')


# 임계값 확인
```

```
if (( $(echo "$CPU_USAGE > $ALERT_THRESHOLD_CPU" | bc -l) )); then
echo "ALERT: High CPU usage: $CPU_USAGE%"
# 알림 전송 로직
fi

if (( $MEMORY_USAGE > $ALERT_THRESHOLD_MEMORY )); then
echo "ALERT: High memory usage: $MEMORY_USAGE bytes"
# 알림 전송 로직
fi

if (( $PEER_COUNT < 5 )); then
echo "WARNING: Low peer count: $PEER_COUNT"
# 알림 전송 로직
fi
```

성능 회귀 테스트

```
func BenchmarkMetricUpdate(b *testing.B) {
counter := metrics.NewCounter()
b.ResetTimer()
b.RunParallel(func(pb *testing.PB) {
for pb.Next() {
counter.Inc(1)
}
```

```
})
}

func BenchmarkRegistryLookup(b *testing.B) {
registry := metrics.NewRegistry()
counter := metrics.NewCounter()
registry.Register("test.counter", counter)
b.ResetTimer()
b.RunParallel(func(pb *testing.PB) {
for pb.Next() {
registry.Get("test.counter")
}
})
}
```

관련 시스템

의존성 관계

1. **Go 런타임**: 런타임 메트릭 제공
2. **운영체제**: 시스템 리소스 정보
3. **HTTP 서버**: 메트릭 노출 엔드포인트
4. **시계열 데이터베이스**: 메트릭 저장 및 분석

상호작용 패턴

- **Application → Metrics**: 메트릭 업데이트
- **Collector → System**: 시스템 정보 수집
- **Registry → Exporter**: 메트릭 데이터 변환
- **Exporter → Client**: 메트릭 데이터 전송
- **Monitor → Alerting**: 임계값 기반 알림

영향 범위

- **성능**: 메트릭 수집으로 인한 1-3% 오버헤드
- **메모리**: 메트릭 데이터 저장으로 인한 메모리 사용
- **네트워크**: 메트릭 전송으로 인한 대역폭 사용
- **스토리지**: 시계열 데이터 저장 공간 필요

이 메트릭 수집 시스템은 geth의 운영 상태를 실시간으로 모니터링하고, 성능 최적화와 문제 예방을 위한 핵심 인프라를 제공합니다. Prometheus, InfluxDB 등 다양한 모니터링 도구와의 통합을 통해 포괄적인 관찰 가능성(Observability)을 구현할 수 있습니다.